
Oliver Krauss

Data that Hurts in SQL

MDB3 | MTD.ba VZ WS26

Hagenberg 01./02.10.2026

HAGENBERG | LINZ | STEYR | WELS



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

You Know SQL. Model This.

One shop catalogue, delivered as 30 CSV files. Four rows from four of them:

Item	File	What the row knows
3D glasses	accessories	variation (White, Black), buy 490, sell 122, two colours, size 1x1, source, style, label themes, villager equippable
angelfish	fish	sell 3000, where (River), shadow (Small), spawn rate, and 24 columns NH Jan ... SH Dec with "4 PM -- 9 AM" or NA
Agent K.K.	music	buy 3200, sell 800, source (K.K. concert), 13 columns in total
acoustic guitar	housewares	variation (Natural, Cherry), DIY yes, body customizable, kit cost 5, HHA concept, interact, outdoor, speaker type

- Every file has its own columns; the 30 files together use **135 different column names**
- The shop wants **one** question answered across all of them: *all items under 500 Bells*

...and Model This.

One villager, one row of `villagers.csv`:

1	Name: Admiral Nature	Species: Bird	Personality: Cranky	Hobby:
2	Birthday: 27-Jan Steep Hill	Catchphrase: aye aye	Favorite Song:	
3	Style 1: Cool Blue	Style 2: Cool	Color 1: Black	Color 2:
4	Wallpaper: dirt-clod wall		Flooring: tatami	
5	Furniture List: 717;1849;7047;2736;787;5970;3449;3622;3802;4106;3438;4029			

- **Style 1, Style 2, Color 1, Color 2**: the same thing twice, numbered
- **Furniture List**: twelve item ids **in one cell**, separated by semicolons
- The game wants two questions answered: *show me Admiral* and *which villagers own item 717?*

Two Schemas, Please

Let's do it together. I draw at the board, you call out the tables. You have MariaDB and what you learned in *Relational Databases*.

1. **One schema for the whole catalogue**, so that *all items under 500 Bells* is a single query. Then a 31st file arrives, **umbrellas.csv**: what changes?
2. **Admiral**, so that *which villagers own item 717?* is a single query. How many tables do we read to show Admiral's page?

We will end up with four different answers for the catalogue. Each one is on the next four slides, with what it costs.

Option 1: One Wide Table

- **items** with the union of all columns: **71 columns** for the four files shown, **135** for all thirty
- The 3D glasses row fills 21 of 71 columns; **50 are NULL**. Agent K.K.: 58 NULL.
- *All items under 500 Bells*: **SELECT name FROM items WHERE sell < 500** works
- **umbrellas.csv** arrives with a new column: **ALTER TABLE** on a table with 20,000 rows, then every **INSERT** in the application changes
- Constraints stop making sense: **kit_cost NOT NULL** is right for housewares and wrong for songs

What it costs

Every new shape of data is a schema change, and the schema can no longer say what a valid row is.

Option 2: A Table per File

```
CREATE TABLE items (  
  id INT PRIMARY KEY, name VARCHAR(60),  
  buy INT, sell INT);  
CREATE TABLE fish (  
  id INT PRIMARY KEY REFERENCES items(id),  
  where_how VARCHAR(20), shadow VARCHAR(10),  
  nh_jan VARCHAR(20), ... , sh_dec VARCHAR(20));  
CREATE TABLE housewares (  
  id INT PRIMARY KEY REFERENCES items(id),  
  diy BOOLEAN, kit_cost INT, ...);  
-- 28 more tables
```

- Common columns once, specific ones per type; *under 500 Bells* is one query on **items**
- *Everything about the angelfish*: a **JOIN** per type, and the code must know which
- A new file: a new table, a new class, a new **JOIN**

What it costs

31 tables describe one thing, and the code knows all 31.

Option 3: A Key-Value Table

```
CREATE TABLE item_attributes (  
  item_id INT REFERENCES items(id),  
  attr_key VARCHAR(40), attr_value VARCHAR(255));  
  
-- all items under 500 Bells  
SELECT i.name FROM items i  
JOIN item_attributes a ON a.item_id = i.id  
WHERE a.attr_key = 'sell'  
      AND CAST(a.attr_value AS INT) < 500;
```

What it costs

You wrote your own type system, badly, inside a relational database.

- Any attribute, no schema change: WordPress stores every post's extras this way (**wp_postmeta**)
- Every value is a **VARCHAR**: no types, no **CHECK**, a **CAST** in every comparison
- One **JOIN** per attribute you want back; no index on "the sell price", there is no such column

Option 4: A JSON Column

```
CREATE TABLE items (id INT PRIMARY KEY,  
    name VARCHAR(60), type VARCHAR(20), sell INT,  
    attributes JSON);
```

```
INSERT INTO items VALUES (1, '3D glasses',  
    'accessory', 122, '{"variation": "White",  
    "colors": ["White","Colorful"],  
    "source": "Able Sisters"}');
```

```
SELECT name FROM items WHERE  
    JSON_VALUE(attributes,'$.source') = 'Able  
    Sisters';
```

- MariaDB has it; the column takes any shape
- Common columns stay typed and indexed; the rest is a blob with a path syntax: no index inside, no validation, a second query language inside the first

This is the interesting one

Half a document database inside a table. **MongoDB is what happens when the whole database is the JSON column**, with indexes, validation and a query language made for it.

Admiral, Normalised

```
CREATE TABLE villagers (id, name, species, ...,
    wallpaper, flooring);
CREATE TABLE villager_styles (villager_id, style)
    ;
CREATE TABLE villager_colors (villager_id, color)
    ;
CREATE TABLE villager_furniture (villager_id,
    item_id);

-- which villagers own item 717?
SELECT v.name FROM villagers v
JOIN villager_furniture f ON f.villager_id = v.id
WHERE f.item_id = 717;
```

- **Style 1/Style 2** and the semicolon list are what 1NF forbids; the fix you learned: a table per repeating group
- *Which villagers own 717?:* one query, correct and fast
- *Show me Admiral:* **four tables** joined, for data never used apart from Admiral

What it costs

Right for the question, wrong for the read. Most reads are “show me this one thing”.

The Catalogue as Documents

One collection, three shapes. Each document carries the fields it has, and no others.

```
1 { "name": "3D glasses", "type": "accessory", "buy": 490, "sell": 122,  
2   "variations": ["White", "Black"], "colors": ["White", "Colorful"],  
3   "source": "Able Sisters", "style": "Active" }  
4 { "name": "angelfish", "type": "fish", "sell": 3000, "where": "River",  
5   "available": { "north": { "May": "4 PM - 9 AM", ..., "Oct": "4 PM - 9 AM" } } }  
6 { "name": "Agent K.K.", "type": "music", "buy": 3200, "sell": 800 }
```

- *Under 500 Bells*: `db.items.find({sell: {$lt: 500}})`, across all shapes
- The angelfish keeps its six months; the other 18 columns do not exist
- `umbrellas.csv` arrives: insert the documents. No **ALTER TABLE**, no new code

Admiral as a Document

```
1 { "name": "Admiral", "species": "Bird", "personality": "Cranky", "hobby": "
  Nature",
2   "birthday": "27-Jan", "catchphrase": "aye aye", "favoriteSong": "Steep
  Hill",
3   "styles": ["Cool", "Cool"], "colors": ["Black", "Blue"],
4   "home": { "wallpaper": "dirt-clod wall", "flooring": "tatami" },
5   "furniture": [717, 1849, 7047, 2736, 787, 5970, 3449, 3622, 3802, 4106, 3
    438, 4029] }
```

- *Show me Admiral*: one document, one read, no **JOIN**
- *Which villagers own 717?*: `db.villagers.find({furniture: 717})`. MongoDB looks inside arrays, and can index them (*Indexes and Schema Validation*)
- The repeating groups became **arrays**, the home became an **embedded document**: the shape of the data is the schema

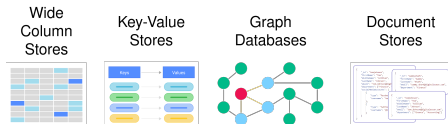
What You Pay for It

- **No JOIN across collections.** Embed (duplicate) or reference (fetch twice): *Document Modelling, today*
- **No schema unless you ask.** "Se11" next to "se11" goes in; validation is opt-in (*Indexes and Schema Validation*)
- **Atomic per document.** Two documents need a transaction (*Transactions and Consistency*)
- **Not SQL.** Filters and pipelines; every SQL clause has a counterpart (*CRUD and Queries, Aggregation I, II*)

The rest of this course

pays that price consciously, then meets two systems that trade differently: NewSQL keeps SQL and ACID at scale, search engines give up both for relevance.

Not Only SQL: the Landscape



- **Document stores:** JSON documents in collections. MongoDB, CouchDB. **This course.**
- **Key-value stores:** one key, one opaque value, in memory. Redis, Memcached. Caches, sessions.
- **Wide-column stores:** rows with any columns, written for one query. Cassandra, HBase.
- **Graph databases:** nodes and relationships, queries walk edges. Neo4j.
- **Search engines:** inverted index, ranked results. Elasticsearch. **This course.**
- **NewSQL:** SQL and ACID on a cluster. CockroachDB, Spanner. **This course.**

The Arc of This Course

1. **SQL hurts on flexible data** (today) → **documents**: *Document Modelling, CRUD and Queries, Aggregation I and II*
2. **The price of documents**: *Indexes and Schema Validation, Transactions and Consistency*
3. **SQL back, at scale**: *NewSQL I and II*, a three-node cluster on your laptop
4. **A problem neither solves well**: *Search I and II*, Elasticsearch
5. **Which one for what**: *Polyglot Persistence*

ACID

- **Atomic**: all of the transaction or none of it
- **Consistent**: constraints hold before and after
- **Isolated**: concurrent transactions do not see each other's half-work
- **Durable**: committed means on disk

What it assumes

One server that sees every write. On one machine ACID is cheap; the moment data lives on several machines every guarantee needs a network round trip, and networks fail.

BASE

- **Basically Available**: the system answers, even while parts of it are down
- **Soft state**: what you read may be in flux; replicas are catching up
- **Eventually consistent**: stop writing and all copies converge

Where you have met it

A friend's post shows on your phone before it shows on their laptop. Two readers of the same feed disagree for a second. Nobody loses money over it, so the system prefers answering over being exactly right.

- BASE is not “no guarantees”; it is **different guarantees**, chosen per system and often per operation
- MongoDB is ACID for one document and lets you choose how much consistency a read or write waits for (*Transactions and Consistency*)

CAP: the Choice During a Partition

- **C**onsistency: every read sees the latest write
- **A**vailability: every request gets an answer
- **P**artition tolerance: the system survives a broken network between its nodes

Not “pick two of three”

Networks partition whether you like it or not, so P is given. The theorem says: **while a partition lasts**, a node that cannot reach the others must either refuse to answer (keep C) or answer with what it has (keep A).

- MongoDB, CockroachDB: a cut-off minority **stops answering writes** (C). You will watch this happen in *NewSQL II*.
- Cassandra, DynamoDB: **keep answering**, reconcile later (A)
- A single-server MariaDB never faces the choice; it is simply down when its machine is

Which Database?

Show of hands, one scenario at a time: document database, single-server SQL, distributed SQL, or “something else from the landscape”?

1. The bank ledger behind a payment app
2. The Animal Crossing catalogue: 30 files, 135 column names, read far more than written
3. A game leaderboard that ranks a million players live
4. Tax records with a legal retention period and audits
5. The event log of 10,000 sensors, each type sending different fields, never updated
6. A user profile with settings, favourites and a list of owned items

Which Database? Answers

1. **SQL**: ACID across accounts, joins for statements, one region: single-server SQL, or distributed SQL if it must survive a data centre
2. **Document**: varying shape, read-heavy, one document per item
3. **Something else**: a sorted set in a key-value store (Redis) answers “rank of player X” in memory; a document or SQL store keeps the player
4. **SQL**: fixed schema, constraints, audit trail; nothing here needs flexibility
5. **Document or wide-column**: varying fields, append-only, huge volume; in this course, MongoDB with a time field
6. **Document**: the Admiral case

The three questions behind every answer

What **shape** is the data? How much **consistency** does a mistake cost? How many **machines** will it need?

Quick Retrieval: Test Yourself

Before the summary: can you answer these from memory?

1. Name the four ways to force the catalogue into MariaDB, and one cost of each.
2. What did Admiral's **Furniture List** cell become in the document, and how do you find every villager who owns item 717?
3. "MongoDB is what happens when ...": finish the sentence.
4. What is the one thing MongoDB is ACID about without you asking?
5. CAP: what does a cut-off MongoDB node do while the partition lasts, and what would Cassandra do?

Think first, then we go through the answers.

Quick Retrieval: Answers

1. One wide table (NULLs, **ALTER TABLE** per new shape), a table per type (a JOIN per type, 31 tables), a key-value table (everything VARCHAR, a JOIN per attribute), a JSON column (a blob without index, validation or a real query language).
2. An array **furniture: [717, ...]; db.villagers.find({furniture: 717})**.
3. ...the whole database is the JSON column, with indexes, validation and a query language made for it.
4. A single document: a write to one document is all-or-nothing.
5. It refuses writes (keeps consistency); Cassandra keeps answering and reconciles later (keeps availability).

Summary: Data that Hurts in SQL

The pain

- Data of varying shape has no good place in a fixed schema
- Every SQL workaround trades away types, constraints or query power
- Normalisation is right for the question and wrong for the read

The document answer

- The shape of the data is the schema; arrays and embedded documents replace junction tables
- You pay with joins, opt-in validation and per-document atomicity

The landscape

- Document, key-value, wide-column, graph, search, NewSQL: each built for one shape of problem

Guarantees

- ACID assumes one server; BASE is a different set of guarantees, not none
- CAP is the choice during a partition: refuse or answer stale

Details: Known and Unknown Unknowns

What we know we do not know

- New fields (a preference, a rating), new types (a JSON blob, an array), new relationships
- Each is a planned schema change: migration script, rollback plan, application update

What we do not know we do not know

- A regulation adds fields; a new device type sends new data; the business model changes
- Netflix went from DVDs to streaming to games; the “film” table did not see it coming

A fixed schema asks you to predict both. A flexible schema lets the data arrive first and the rules follow.

Details: Document Store or Key-Value Store?

- A key-value store maps a key to an **opaque** value: the database does not look inside, so it cannot search by a field or index one
- A document store keeps the value **structured**: fields can be queried, indexed, validated and updated in place
- Every document has a key (**_id**); so a document store is a key-value store that understands its values
- The line blurs: Redis has JSON and search modules; MongoDB can be used as a plain key-value cache

Details: Wide-Column Stores

- Rows in tables, but a row may have any columns; storage is by column, not by row
- No joins: data is written already denormalised for the query that will read it (query-driven design)
- Cassandra, HBase, ScyllaDB; time series, IoT, write-heavy logs

Row oriented:

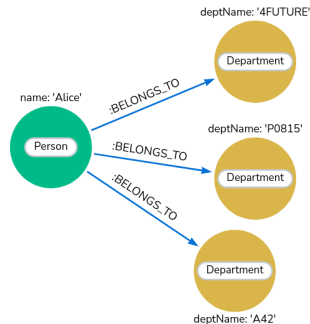
ID	Name	Lecture	Grade
1	Max	MDB	3
2	Ada	MDB	1
3	Lex	AIN	1

Column oriented:

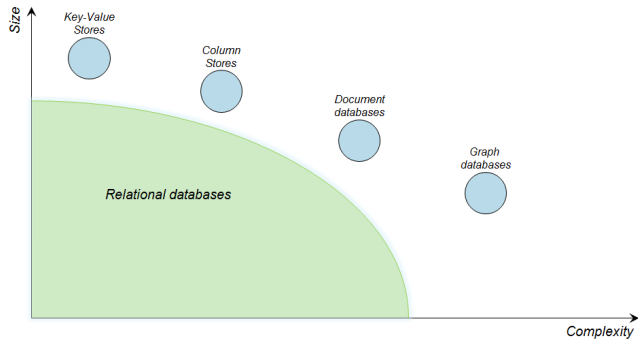
ID	Name	ID	Lecture	ID	Grade
1	Max	1	MDB	1	3
2	Ada	2	MDB	2	1
3	Lex	3	AIN	3	1

Details: Graph Databases

- Nodes carry data, relationships carry data and direction; a query walks the graph
- “Friends of friends who bought this” is three hops, not three self-joins
- Neo4j (Cypher), Amazon Neptune; social graphs, fraud detection, knowledge graphs, recommendations
- As general as the relational model; the win is in relationship-heavy queries, the loss in bulk analytics



Details: Data Complexity against Query Complexity



- Left to right: how complex is the data; bottom to top: how complex are the queries
- Key-value stores sit bottom left, graph databases top right; document stores cover the middle

Details: Polyglot Persistence in the Wild

- **Web shop:** PostgreSQL for orders and payments, MongoDB for the product catalogue, Redis for carts and sessions, Elasticsearch for search
- **Social network:** Cassandra for the timeline, Neo4j for the friend graph, Redis for presence
- **IoT platform:** a time-series database for measurements, MongoDB for device documents, Elasticsearch for log analytics

The rule

Nobody uses one database for everything any more. The skill is knowing which shape of problem each one is built for, and what its guarantees cost. The closing session, *Polyglot Persistence*, is a design exercise on exactly this.

Details: BASE in Practice

- **Social feed**: a post reaches some followers before others; everybody sees it eventually
- **Shopping cart across devices**: the phone and the laptop disagree for a moment, then merge
- **Content delivery network**: an edge server hands out the old page until the new one propagates

What they have in common

A stale answer costs less than no answer. Where that is false (money, stock, seats), the system uses ACID for that part.

Details: RDBMS, NoSQL, NewSQL

	RDBMS	Document store	NewSQL
Guarantees	ACID	ACID per document, tunable	ACID, serializable
Schema	fixed	flexible, opt-in validation	fixed
Scaling	up (bigger machine)	out (sharding built in)	out (ranges, consensus)
Joins	yes	limited (\$lookup)	yes
Query language	SQL	MQL, pipelines	SQL
Examples	MariaDB, PostgreSQL	MongoDB, CouchDB	CockroachDB, Spanner

NewSQL is not “NoSQL with SQL on top”: a relational database whose storage is a replicated, sharded key-value layer. *NewSQL* / explains how.